Universidad Autónoma de Bucaramanga (UNAB)
Facultad de Ingeniería – Ingeniería de Sistemas

Investigación de Operaciones I

Cobertura de clientes con antenas de telefonía: formulación Set Cover, solución exacta y Algoritmo Genético

María Isabel Vargas Mendoza
Oscar Iván Vera Puerto

Bucaramanga, Colombia
Noviembre de 2026

Tabla de contenido

1 Introducción **2**
1.1 Objetivos . . . 2
1.2 Estructura del documento . . . 2

2 Marco teórico **3**
2.1 El problema de cobertura de conjuntos (Set Cover) . . . 3
2.2 Complejidad computacional . . . 3
2.3 Programación lineal entera binaria . . . 3
2.4 Métodos exactos: ramificación y acotamiento . . . 3
2.5 Metaheurísticas y Algoritmos Genéticos . . . 4
2.6 Manejo de restricciones en el Algoritmo Genético . . . 4
2.7 Solver utilizado . . . 4

3 Metodología **5**
3.1 Descripción de los datos . . . 5
3.2 Formulación matemática . . . 5
3.3 Solución exacta . . . 6
3.4 Algoritmo Genético . . . 6
3.5 Diseño experimental . . . 8
3.6 Interfaz visual . . . 8

4 Resultados **8**
4.1 Entorno de ejecución . . . 8
4.2 Solución exacta . . . 8
4.3 Heurística voraz de referencia . . . 9
4.4 Algoritmo Genético . . . 9
4.5 Comparación entre el método exacto y el Algoritmo Genético . . . 10
4.6 Análisis . . . 11

5 Conclusiones **12**

Anexos y apéndices **13**
Anexo A. Instancia entregada por el docente . . . 13
Apéndice A. Resultados del Algoritmo Genético por corrida . . . 13
Apéndice B. Mejor solución entera del método exacto . . . 14
Apéndice C. Código fuente . . . 14
Apéndice D. Reproducibilidad y repositorio . . . 17

1 Introducción

El despliegue de una red de telefonía móvil exige decidir en cuáles de un conjunto de ubicaciones candidatas se instalarán antenas para que todos los usuarios queden dentro del área de cobertura de, al menos, una de ellas. Cada ubicación tiene un costo de instalación distinto y cubre a un subconjunto particular de clientes. El objetivo de la empresa es doble: no dejar a ningún cliente sin servicio y hacerlo con el menor costo total posible.

Este problema es una instancia clásica del *problema de cobertura de conjuntos* (*Set Cover Problem*, SCP), uno de los 21 problemas NP-completos de Karp [1]. Aparece, además de en telecomunicaciones, en la asignación de personal, la localización de servicios de emergencia, la selección de pruebas de software y el diseño de circuitos. Aunque su enunciado es sencillo, su dificultad computacional crece con el tamaño de la instancia: no se conoce un algoritmo que lo resuelva de forma eficiente en el caso general [2], y resulta difícil incluso de aproximar mejor que un factor logarítmico [4].

En este trabajo se considera una instancia con 500 clientes y 500 ubicaciones candidatas, cuyos subconjuntos de cobertura se entregan en una matriz binaria y cuyos costos se entregan en un archivo aparte. El trabajo plantea el modelo de programación lineal entera binaria del Set Cover y lo resuelve mediante un Algoritmo Genético (AG) diseñado para este problema. Además, para disponer de un punto de referencia, se resuelve el mismo modelo con un solver exacto y se comparan ambos enfoques en calidad de la solución y tiempo de cómputo.

1.1 Objetivos

Objetivo general. Determinar el conjunto de antenas de menor costo que cubra a los 500 clientes, modelando el problema como Set Cover y resolviéndolo con un Algoritmo Genético.

Objetivos específicos.

- Formular el modelo de programación lineal entera binaria del problema, definiendo variables, función objetivo y restricciones.
- Diseñar e implementar un Algoritmo Genético con operadores de selección, cruce, mutación y reparación adecuados para el Set Cover.
- Resolver el modelo con un solver exacto y usar su resultado como referencia.
- Comparar ambos métodos en calidad de solución, tiempo de cómputo y comportamiento, y analizar el efecto de los parámetros del AG.

1.2 Estructura del documento

La sección 2 presenta el marco teórico. La sección 3 describe la metodología: datos, modelo matemático, método exacto y Algoritmo Genético. La sección 4 reporta y analiza

los resultados. La sección 5 presenta las conclusiones.

2 Marco teórico

2.1 El problema de cobertura de conjuntos (Set Cover)

Sea U un universo de m elementos y sea $\mathcal{S} = \{S_1, \ldots, S_n\}$ una colección de subconjuntos de U, cada uno con un costo $c_j > 0$. El problema de cobertura de conjuntos consiste en elegir una subcolección $\mathcal{C} \subseteq \mathcal{S}$ tal que todo elemento de U pertenezca a al menos un subconjunto elegido, de modo que la suma de los costos de $\mathcal{C}$ sea mínima. Cuando todos los costos son iguales, el problema se reduce a minimizar el número de subconjuntos seleccionados (versión sin pesos); cuando los costos difieren, se habla de la versión ponderada, que es la que se estudia aquí.

En el contexto de telecomunicaciones, el universo U es el conjunto de clientes, cada subconjunto S_j es el conjunto de clientes que quedan cubiertos por una antena instalada en la ubicación j, y c_j es el costo de instalar esa antena.

2.2 Complejidad computacional

La versión de decisión del Set Cover fue una de las 21 problemas NP-completos identificados por Karp [1]. Por tanto, salvo que P = NP, no existe un algoritmo que lo resuelva en tiempo polinomial para toda instancia [2]. Además, Feige [4] mostró que, bajo supuestos estándar de complejidad, no es posible aproximarlo con un factor mejor que $(1 - \varepsilon) \ln m$. La heurística voraz de Chvátal [3], que en cada paso elige el subconjunto con menor costo por elemento nuevo cubierto, alcanza una garantía de $H(d) \leq 1 + \ln d$ veces el óptimo, donde d es el tamaño del subconjunto más grande; esta es esencialmente la mejor garantía posible en el peor caso.

2.3 Programación lineal entera binaria

El Set Cover se formula de manera natural como un problema de programación lineal entera binaria (PLEB): las variables de decisión toman únicamente los valores 0 o 1, y tanto la función objetivo como las restricciones son lineales [10]. La relajación lineal del problema reemplaza la condición $x_j \in \{0, 1\}$ por $0 \leq x_j \leq 1$; su valor óptimo es una cota inferior del costo óptimo entero, y es la base de los métodos de solución exacta.

2.4 Métodos exactos: ramificación y acotamiento

El método de ramificación y acotamiento (*branch and bound*) [11] resuelve problemas enteros explorando un árbol de subproblemas. En cada nodo se resuelve la relajación lineal; si su valor es peor que la mejor solución entera conocida (la *incumbente*), el nodo se descarta; si la solución es entera, se actualiza la incumbente; en otro caso se ramifica sobre una variable fraccionaria. Los solvers modernos añaden planos de corte, heurísticas

de primal y preprocesamiento. En el caso del Set Cover, Caprara, Fischetti y Toth [6] revisan los algoritmos exactos y heurísticos y muestran que, a medida que crece el número de filas y columnas, los métodos exactos requieren tiempos que pueden volverse prohibitivos aun con solvers de alto desempeño.

2.5 Metaheurísticas y Algoritmos Genéticos

Una *metaheurística* es un marco de búsqueda general que no garantiza optimalidad, pero que suele entregar soluciones de buena calidad en tiempos razonables. Los Algoritmos Genéticos, propuestos por Holland [7] y popularizados por Goldberg [8], se inspiran en la evolución natural. Mantienen una *población* de soluciones candidatas (*cromosomas*) que evoluciona durante varias *generaciones* mediante cuatro mecanismos [9]:

- **Evaluación (fitness):** una función que mide la calidad de cada individuo.
- **Selección:** los individuos mejores tienen mayor probabilidad de reproducirse; una forma común es la selección por torneo, donde se elige al mejor de k individuos tomados al azar.
- **Cruce (*crossover*):** combina el material genético de dos padres para producir hijos; en el cruce uniforme, cada gen del hijo se toma de uno de los dos padres con probabilidad 1/2.
- **Mutación:** modifica aleatoriamente algunos genes para mantener la diversidad y evitar la convergencia prematura.

Se suele añadir *elitismo*, que copia los mejores individuos a la siguiente generación sin cambios, de modo que la mejor solución encontrada nunca se pierde.

2.6 Manejo de restricciones en el Algoritmo Genético

Un cromosoma binario cualquiera no tiene por qué ser factible: puede dejar clientes sin cubrir. Hay dos estrategias habituales. La primera es *penalizar* el fitness de los individuos infactibles; es simple, pero la población puede pasar mucho tiempo explorando regiones infactibles. La segunda, empleada por Beasley y Chu [5] en su algoritmo genético para el Set Cover, es *reparar* el cromosoma: se agregan columnas hasta lograr factibilidad y luego se eliminan las columnas redundantes. Esta es la estrategia adoptada en este trabajo, pues garantiza que todo individuo evaluado sea una solución válida.

2.7 Solver utilizado

Para la solución exacta se usó HiGHS [12], un solver de código abierto para programación lineal y entera mixta, accedido a través de la función `milp` de SciPy [13].

3 Metodología

3.1 Descripción de los datos

Se dispone de dos archivos. El primero, `set_cover_500x500.csv`, es una matriz de 500×500 con valores 0 y 1, donde la entrada de la fila i y la columna j vale 1 si la antena ubicada en j cubre al cliente i. El segundo, `Costo_S.xlsx`, contiene el costo de instalación de cada una de las 500 antenas.

La matriz tiene una densidad de 0,0999, es decir, cerca del 10 % de las entradas son unos. Cada cliente queda cubierto por entre 32 y 70 antenas, y cada antena cubre entre 33 y 66 clientes. Los costos van de 2 000 a 3 998, con un promedio de 3 033,6. La Figura 1 muestra la distribución de los costos y de la cobertura por cliente. Como todo cliente tiene al menos 32 antenas que lo cubren, la instancia es factible (por ejemplo, instalando todas las antenas).

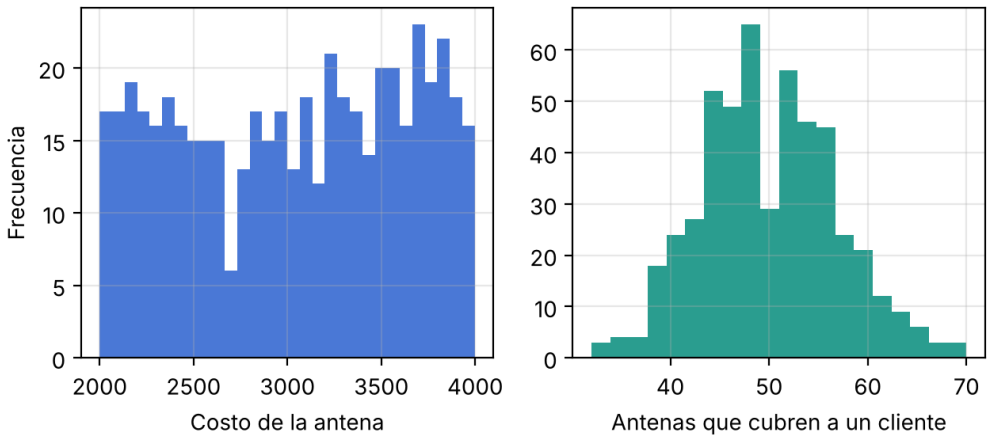


Figura 1: Distribución de los costos de las antenas y del número de antenas que cubren a cada cliente.

3.2 Formulación matemática

Conjuntos e índices. $I = \{1, \ldots, 500\}$ es el conjunto de clientes, indexado por i; $J = \{1, \ldots, 500\}$ es el conjunto de ubicaciones de antenas, indexado por j.

Parámetros.

- $a_{ij} \in \{0, 1\}$: vale 1 si la antena j cubre al cliente i y 0 en caso contrario.
- $c_j > 0$: costo de instalar la antena j.

Variables de decisión.

$$x_j = \begin{cases} 1 & \text{si se instala una antena en la ubicación } j, \\ 0 & \text{en caso contrario,} \end{cases} \qquad j \in J.$$

Modelo.

$$\min \; Z = \sum_{j \in J} c_j \, x_j \tag{1}$$

$$\text{s.a.} \quad \sum_{j \in J} a_{ij} \, x_j \geq 1 \qquad \forall \, i \in I \tag{2}$$

$$x_j \in \{0, 1\} \qquad \forall \, j \in J \tag{3}$$

Significado de cada componente. La función objetivo (1) es el costo total de las antenas instaladas, que se desea minimizar. Las restricciones (2) garantizan la cobertura: para cada cliente i, la suma $\sum_j a_{ij} x_j$ cuenta cuántas antenas instaladas lo cubren, y debe ser al menos 1. Las restricciones (3) indican que cada ubicación se usa o no se usa; no se pueden instalar fracciones de antena. El modelo tiene 500 variables binarias y 500 restricciones de cobertura.

3.3 Solución exacta

El modelo (1)–(3) se implementó en Python y se resolvió con el solver HiGHS [12] mediante la función `scipy.optimize.milp`. Se fijó un límite de tiempo de 600 segundos y una brecha relativa de optimalidad de 0. Se registraron el costo de la mejor solución entera, la cota inferior del solver, la brecha entre ambos y el tiempo de cómputo.

3.4 Algoritmo Genético

Codificación. Cada individuo es un vector binario $\mathbf{x} = (x_1, \ldots, x_{500})$ con la misma interpretación que las variables del modelo: $x_j = 1$ si se instala la antena j.

Reparación. Dado un vector $\mathbf{x}$ cualquiera, se aplica el procedimiento de reparación de dos fases, inspirado en Beasley y Chu [5]:

1. **Fase de cobertura:** mientras haya clientes sin cubrir, se agrega la antena que minimiza la razón c_j / n_j, donde n_j es el número de clientes sin cubrir que la antena j cubriría. Esto es el criterio voraz de Chvátal [3].

2. **Fase de eliminación de redundancias:** se recorren las antenas instaladas de la más cara a la más barata y se elimina cada una cuya salida deje a todos sus clientes con al menos otra antena.

Fitness. Como todo individuo reparado es factible, el fitness es simplemente el costo $f(\mathbf{x}) = \sum_j c_j x_j$, que se minimiza.

Inicialización. Cada individuo de la población inicial se genera activando cada gen con probabilidad $p_{\text{init}} = 0{,}05$ y reparando el resultado. Esto produce individuos diversos y factibles desde el comienzo.

Selección. Se usa selección por torneo de tamaño $k = 3$: se toman k individuos al azar y se elige el de menor costo.

Cruce. Con probabilidad $p_c = 0{,}9$, los dos padres se combinan mediante cruce uniforme: cada gen del hijo se toma de un padre u otro con probabilidad 1/2. Con probabilidad $1 - p_c$ el hijo es copia del primer padre.

Mutación. Cada gen del hijo se invierte de forma independiente con probabilidad $p_m = 1/n = 0{,}002$, de modo que en promedio se muta un gen por hijo. Tras la mutación el hijo se repara.

Elitismo y criterio de parada. Los 2 mejores individuos pasan sin cambios a la siguiente generación. El algoritmo se detiene al llegar a 300 generaciones o cuando el mejor costo no mejora durante 80 generaciones consecutivas.

Parámetros. La Tabla 1 resume los parámetros de la configuración base y las variantes que se estudiaron.

Tabla 1: Parámetros del Algoritmo Genético.

Parámetro	Base	Población pequeña	Población grande	Mutación alta
Tamaño de población	100	30	300	100
Probabilidad de cruce	0,9	0,9	0,9	0,9
Probabilidad de mutación por gen	1/500	1/500	1/500	5/500
Tamaño del torneo	3	3	3	3
Élite	2	2	2	2
Máx. generaciones	300	300	300	300
Paciencia (generaciones)	80	80	80	80

Algoritmo 1 Algoritmo Genético con reparación para el Set Cover

Generar población inicial de N individuos y repararlos
Evaluar el costo de cada individuo
for $g = 1, \ldots, G_{\max}$ **do**
    Copiar los E mejores individuos a la nueva población (elitismo)
    while la nueva población tenga menos de N individuos **do**
        Seleccionar dos padres por torneo
        Con probabilidad p_c aplicar cruce uniforme; si no, copiar el primer padre
        Mutar cada gen con probabilidad p_m
        Reparar el hijo y agregarlo a la nueva población
    Evaluar la nueva población y actualizar la mejor solución
    if no hubo mejora en P generaciones **then**
        parar
return mejor solución encontrada

3.5 Diseño experimental

Los algoritmos se implementaron en Python con NumPy y SciPy [13]. El AG es estocástico, por lo que un único resultado no es representativo: cada configuración de la Tabla 1 se ejecutó 10 veces con semillas distintas (0 a 9). Se registraron el costo final, el número de antenas, el número de generaciones y el tiempo de cómputo. Además se ejecutó una configuración *extendida* (población de 200, mutación $2/n$, hasta 3 000 generaciones y paciencia de 400) con otras 10 semillas, para estudiar el efecto de una búsqueda más larga. Como referencias adicionales se calcularon la heurística voraz de Chvátal y la cota inferior de la relajación lineal. Para comparar con el método exacto se calculó la brecha relativa

$$\text{Brecha} = \frac{Z_{\text{AG}} - Z_{\text{ref}}}{Z_{\text{ref}}} \times 100\,\%,$$

donde Z_{ref} es el costo de la mejor solución entera del solver exacto.

3.6 Interfaz visual

Se desarrolló una aplicación web con Streamlit que permite cargar los datos, ejecutar el solver exacto y el AG con parámetros ajustables, ver las curvas de convergencia y comparar ambos métodos en costo, tiempo y antenas seleccionadas.

4 Resultados

4.1 Entorno de ejecución

Los algoritmos se implementaron en Python con NumPy y SciPy [13]. El solver exacto es HiGHS [12] con un límite de tiempo de 600 segundos. El Algoritmo Genético se ejecutó con semillas distintas para cada corrida. Los tiempos de cómputo dependen del equipo en que se ejecuta el código, por lo que deben leerse como una comparación relativa entre métodos y no como valores absolutos.

4.2 Solución exacta

El solver HiGHS no logró demostrar la optimalidad en los 600 segundos permitidos. Terminó con la mejor solución entera encontrada, de **costo 50 119** y 22 antenas, una cota inferior (*dual bound*) de 30 817 y una brecha de optimalidad de 38,5 %. Exploró 19 036 nodos del árbol de ramificación. La relajación lineal del modelo, que se resuelve en menos de un segundo, da una cota inferior de 27 860. Es decir, el costo óptimo está entre 30 817 y 50 119, según lo que el solver pudo demostrar.

Este comportamiento es el esperado para una instancia densa de 500×500: cada cliente tiene entre 32 y 70 antenas que lo cubren, de modo que hay muchísimas combinaciones casi equivalentes y la cota inferior mejora muy lentamente, como lo discuten Caprara, Fischetti y Toth [6].

4.3 Heurística voraz de referencia

Como referencia adicional se aplicó la heurística voraz de Chvátal [3], que es la fase de cobertura de la reparación del Algoritmo Genético. Obtuvo un costo de 52 063 con 23 antenas en menos de 0,01 segundos.

4.4 Algoritmo Genético

La Tabla 2 resume 10 corridas independientes por configuración. La configuración *extendida* usa población de 200, mutación $2/n$, hasta 3 000 generaciones y paciencia de 400 generaciones.

Tabla 2: Resultados del Algoritmo Genético (10 corridas por configuración). La brecha se mide contra la mejor solución entera del solver exacto (50 119); un valor negativo significa que el AG obtuvo menor costo.

Configuración	Mejor	Media	Desv.	Peor	Tiempo (s)	Brecha mejor	Brecha me
Base ($N = 100$, $p_m = 1/n$)	48 878	51 622	1 500	53 807	2,1	−2,5 %	+3,0
Población pequeña ($N = 30$)	50 914	54 134	1 712	57 351	0,7	+1,6 %	+8,0
Población grande ($N = 300$)	49 774	50 809	992	53 414	7,3	−0,7 %	+1,4
Mutación alta ($p_m = 5/n$)	49 678	51 452	1 328	53 976	3,1	−0,9 %	+2,7
Extendida ($N = 200$, $p_m = 2/n$)	49 678	50 436	606	52 038	16,6	−0,9 %	+0,6

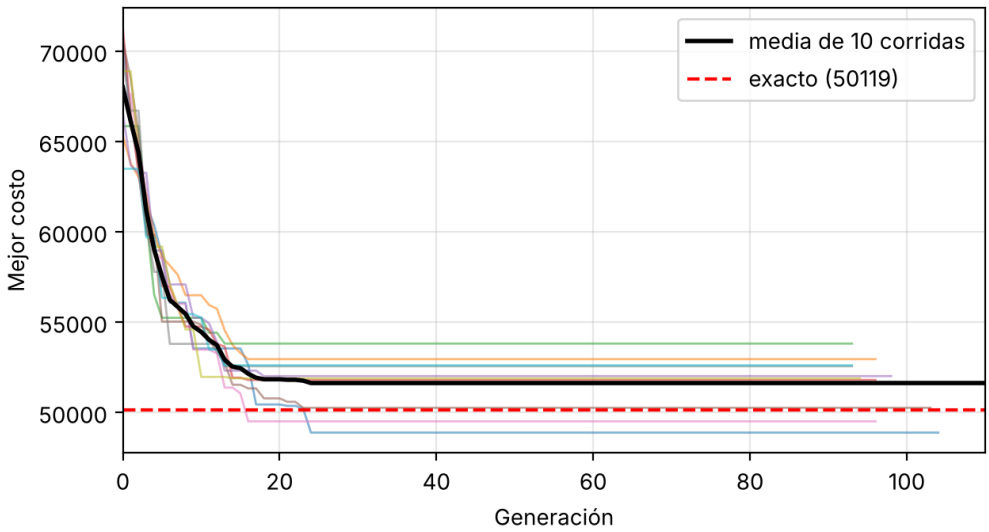


Figura 2: Convergencia del Algoritmo Genético (configuración base). Las líneas delgadas son las 10 corridas; la línea gruesa es su promedio; la línea roja punteada es el costo de la mejor solución del solver exacto.

La Figura 2 muestra que el AG mejora rápidamente en las primeras 20 a 25 generaciones, partiendo de costos cercanos a 68 000, y después se estanca: la mayoría de las corridas deja de mejorar y se detiene por el criterio de paciencia hacia la generación 100.

Esto indica convergencia prematura: la población pierde diversidad y el cruce deja de producir mejoras.

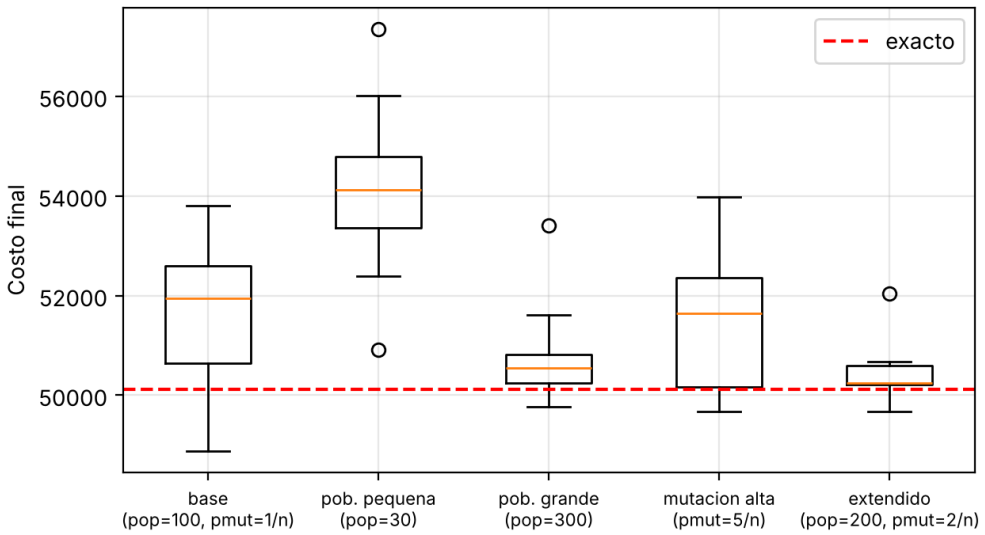


Figura 3: Distribución del costo final por configuración del AG (10 corridas cada una). La línea roja es la mejor solución del solver exacto.

4.5 Comparación entre el método exacto y el Algoritmo Genético

La Figura 4 compara los costos de todos los métodos y la Tabla 3 los resume junto con sus tiempos.

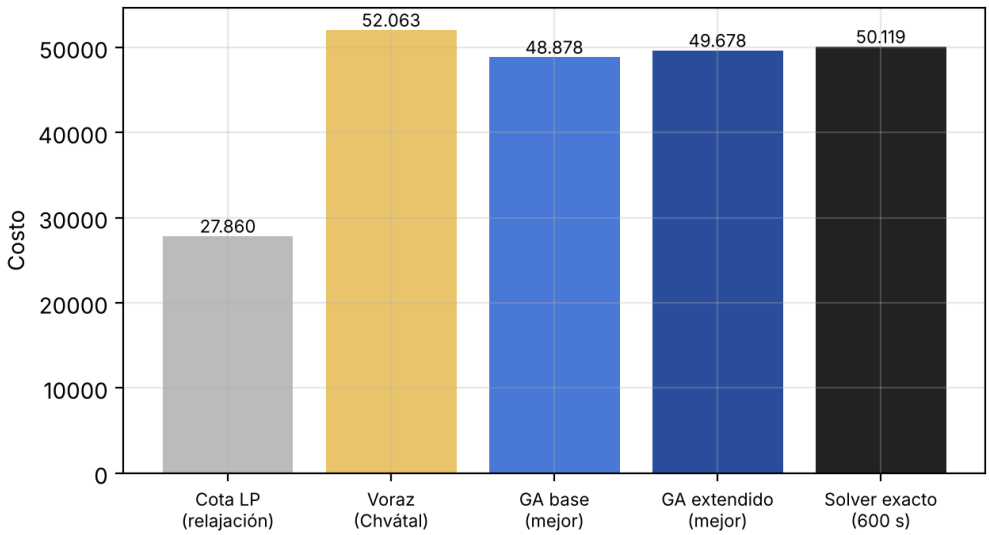


Figura 4: Costo obtenido por cada método.

Tabla 3: Comparación de métodos.

Método	Costo	Antenas	Tiempo (s)
Cota inferior (relajación lineal)	27 860	–	0,2
Cota inferior del solver (600 s)	30 817	–	600
Solver exacto (mejor entera)	50 119	22	600
Heurística voraz	52 063	23	$< 0{,}01$
AG base (mejor de 10)	48 878	–	2,1 (promedio)
AG base (media de 10)	51 622	23,6	2,1
AG extendido (media de 10)	50 436	23,0	16,6

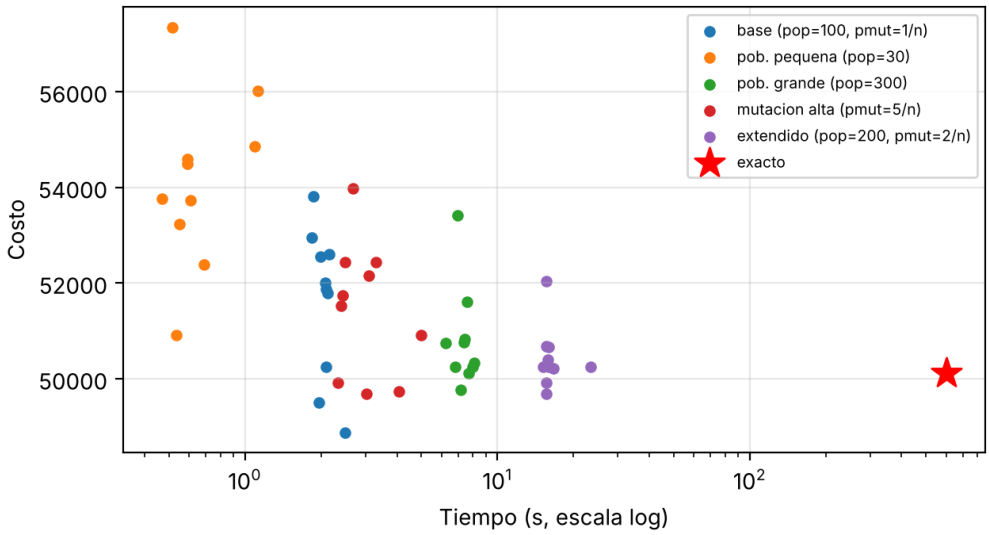


Figura 5: Costo frente al tiempo de cómputo (escala logarítmica en el tiempo). La estrella roja es el solver exacto.

4.6 Análisis

Calidad de la solución. La mejor solución encontrada por el AG (48 878, en la configuración base) es 2,5 % más barata que la mejor solución entera que el solver encontró en 600 segundos, y 6,1 % más barata que la heurística voraz. Sin embargo, el AG es estocástico: en la configuración base el costo medio es 3,0 % mayor que el del solver y varía entre 48 878 y 53 807. La configuración extendida es más estable (desviación de 606) y su media queda a 0,6 % del solver.

Tiempo de cómputo. El AG base necesita unos 2 segundos por corrida; el solver agotó 600 segundos sin demostrar optimalidad. El AG llega a soluciones de calidad comparable en una fracción del tiempo.

Efecto de los parámetros. Una población pequeña ($N = 30$) empeora claramente los resultados (media +8,0 % sobre el solver) porque explora poco. Una población mayor reduce la media y la dispersión, pero cuesta más tiempo (7,3 s con $N = 300$). Aumentar la mutación a $5/n$ mejora levemente el mejor resultado sin costo de tiempo importante.

La configuración extendida logra la mejor media y la menor dispersión al precio de unos 17 segundos.

¿El AG alcanzó el óptimo? No se puede afirmar. El solver no demostró la optimalidad, y su mejor cota inferior es 30 817, de modo que el costo óptimo está entre 30 817 y 48 878 (la mejor solución conocida). Lo que sí se puede afirmar es que el AG obtuvo soluciones factibles de menor costo que las del solver en el tiempo límite. Para saber qué tan lejos está del óptimo haría falta un solver con más tiempo o una cota inferior más fuerte.

5 Conclusiones

1. El problema de cobertura de antenas se formuló como un modelo de programación lineal entera binaria con 500 variables y 500 restricciones de cobertura, que minimiza el costo total de instalación y exige que cada cliente sea cubierto por al menos una antena.

2. Para esta instancia densa de 500×500, el solver exacto HiGHS no logró demostrar la optimalidad en 600 segundos: terminó con una solución de costo 50 119 y una brecha de 38,5 % respecto a su mejor cota inferior. Esto ilustra por qué el Set Cover, por ser NP-difícil, es costoso de resolver de forma exacta cuando crece el tamaño de la instancia.

3. El Algoritmo Genético con reparación diseñado encontró soluciones factibles en pocos segundos. Su mejor resultado (48 878) fue 2,5 % más barato que la mejor solución del solver en el tiempo límite, y 6,1 % más barato que la heurística voraz. El AG es entonces una alternativa práctica cuando el tiempo de cómputo es limitado.

4. El AG es estocástico: el costo varía de una corrida a otra (desviación de unos 1 500 en la configuración base). Aumentar la población o usar una búsqueda más larga reduce esa variabilidad (desviación de 606 en la configuración extendida), a cambio de más tiempo.

5. El AG mostró convergencia prematura: mejora rápido en las primeras generaciones y luego se estanca. Estrategias como mayor mutación, mecanismos de diversidad o búsqueda local podrían mejorar los resultados.

6. No es posible afirmar que el AG alcanzó el óptimo, porque el solver no lo demostró. El óptimo está entre 30 817 y 48 878.

Limitaciones y trabajo futuro

El estudio usa una sola instancia, un solo solver y un límite de 600 segundos. Trabajo futuro: dar más tiempo al solver o usar la solución del AG como punto de partida del

método exacto, probar otros operadores de cruce y de selección, incorporar búsqueda local, y evaluar el método en más instancias, por ejemplo las de la OR-Library, con pruebas estadísticas entre configuraciones.

Anexos y apéndices

En el **Anexo** se describe el material entregado por el docente (los datos de la instancia). Los **Apéndices** contienen material elaborado por los autores: resultados completos, la solución exacta, el código fuente y las instrucciones para reproducir todo.

Anexo A. Instancia entregada por el docente

La instancia consiste en una matriz binaria A de 500×500 (filas: clientes; columnas: antenas), donde $a_{ij} = 1$ si la antena j cubre al cliente i, y un vector de costos c_j por antena. La Tabla 4 resume sus características.

Tabla 4: Características de la instancia

Característica	Valor
Clientes (restricciones)	500
Antenas (variables)	500
Densidad de la matriz (proporción de unos)	0.100
Antenas que cubren a cada cliente (mín. / media / máx.)	32 / 49.9 / 70
Clientes que cubre cada antena (mín. / media / máx.)	33 / 49.9 / 66
Costo por antena (mín. / media / máx.)	2 000 / 3 034 / 3 998
Costo de instalar todas las antenas	1 516 821

Apéndice A. Resultados del Algoritmo Genético por corrida

Tabla 5: Configuración base (población 100, mutación $1/n$): resultado de cada corrida

Semilla	Costo	Antenas	Generaciones	Tiempo (s)
0	48 878	22	104	2.5
1	52 945	25	96	1.8
2	53 807	24	93	1.9
3	51 793	24	96	2.1
4	52 007	24	98	2.1
5	50 253	23	103	2.1
6	49 501	22	96	2.0
7	52 551	24	93	2.0
8	51 881	24	94	2.1
9	52 603	24	93	2.2
Mejor / media / peor			48 878 / 51 622 / 53 807	

Tabla 6: Configuración extendida (población 200, mutación $2/n$): resultado de cada corrida

Semilla	Costo	Antenas	Generaciones	Tiempo (s)
100	50 209	22	449	16.6
101	50 253	23	617	23.5
102	50 403	23	423	15.8
103	50 681	23	421	15.7
104	50 669	23	425	16.0
105	50 253	23	416	15.2
106	49 678	23	418	15.6
107	50 253	23	419	16.0
108	49 920	23	421	15.6
109	52 038	24	415	15.7
Mejor / media / peor			49 678 / 50 436 / 52 038	

Apéndice B. Mejor solución entera del método exacto

HiGHS terminó por límite de tiempo (600 s), por lo que esta solución es la mejor encontrada y no está certificada como óptima: costo 50 119, cota inferior 30 817 y brecha 38.51 %. Las 22 antenas seleccionadas (numeradas desde 1) son:

> 13, 19, 44, 45, 73, 79, 94, 200, 210, 277, 307, 334, 357, 366, 401, 404, 411, 432, 444, 450, 473, 475.

Apéndice C. Código fuente

Se incluyen los tres módulos centrales. El código completo (experimentos, figuras e interfaz) está en el repositorio indicado en el Apéndice D.

Código 1: Carga de datos (`setcover_datos.py`)

```python
"""Carga de datos del proyecto Set Cover (antenas / clientes)."""
import os
import numpy as np
import pandas as pd
import openpyxl

BASE = os.path.dirname(os.path.abspath(__file__))

def cargar_datos(ruta_csv=None, ruta_xlsx=None):
    """Devuelve A (clientes x antenas, 0/1) y c (costo por antena)."""
    ruta_csv = ruta_csv or os.path.join(BASE, "datos", "set_cover_500x500.csv")
    ruta_xlsx = ruta_xlsx or os.path.join(BASE, "datos", "Costo_S.xlsx")
    A = pd.read_csv(ruta_csv).values.astype(np.int8)      # fila i = cliente, columna j = antena
    ws = openpyxl.load_workbook(ruta_xlsx, data_only=True).active
    fila_costos = list(ws.iter_rows(values_only=True))[1]
    c = np.array([v for v in fila_costos[1:] if v is not None], dtype=float)
    assert A.shape[1] == len(c), "numero de antenas distinto al numero de costos"
    return A, c
```

Código 2: Solución exacta con HiGHS (solucion_exacta.py)

```python
"""Solucion exacta del Set Cover como PLE binario con HiGHS (scipy.optimize.milp)."""
import time
import numpy as np
from scipy.optimize import milp, LinearConstraint, Bounds
from setcover_datos import cargar_datos

def resolver_exacto(A, c, limite_tiempo=600, gap_rel=0.0, verbose=False):
    m, n = A.shape
    t0 = time.time()
    res = milp(c=c,
               constraints=LinearConstraint(A, lb=np.ones(m), ub=np.inf),  # cada cliente cubierto >= 1 vez
               integrality=np.ones(n),                                     # x_j entero
               bounds=Bounds(0, 1),                                        # x_j binario
               options={"time_limit": limite_tiempo, "mip_rel_gap": gap_rel, "disp": verbose})
    t = time.time() - t0
    x = np.round(res.x).astype(int) if res.x is not None else None
    return {"x": x, "costo": res.fun, "tiempo": t, "estado": res.message,
            "optimo": res.status == 0, "cota": getattr(res, "mip_dual_bound", None),
            "gap": getattr(res, "mip_gap", None)}

if __name__ == "__main__":
    A, c = cargar_datos()
    r = resolver_exacto(A, c, verbose=True)
    print("Estado:", r["estado"])
    print("Costo optimo:", r["costo"], "| antenas:", int(r["x"].sum()), "| tiempo: %.2f s" % r["tiempo"])
    print("Cobertura valida:", bool((A @ r["x"] >= 1).all()))
```

Código 3: Algoritmo Genético con reparación (algoritmo_genetico.py)

```python
"""Algoritmo genetico (GA) para el Set Cover con costos.

Representacion : cromosoma binario de longitud n (x_j = 1 si se instala la antena j).
Fitness        : costo total de la solucion YA REPARADA (minimizar).
Inicializacion : individuos aleatorios (prob. p_init de cada gen) + reparacion.
Seleccion      : torneo de tamano k.
Cruce          : uniforme (cada gen del hijo viene de un padre al azar), prob. p_cruce.
Mutacion       : bit-flip independiente por gen con prob. p_mut.
Reparacion     : (1) agrega antenas por la mejor razon costo / clientes nuevos cubiertos
                   hasta cubrir a todos; (2) elimina antenas redundantes, de la mas cara a la mas barata.
Elitismo       : los n_elite mejores pasan sin cambios.
Parada         : max_gen generaciones, o paciencia sin mejora, o limite de tiempo.
"""
import time
import numpy as np

PARAMS_DEFECTO = dict(pop=100, max_gen=300, p_cruce=0.9, p_mut=None, torneo=3,
                      n_elite=2, p_init=0.05, paciencia=80, limite_tiempo=None)

class SetCoverGA:
    def __init__(self, A, c, seed=0, **params):
        self.A = np.asarray(A, dtype=np.float32)          # clientes x antenas
        self.c = np.asarray(c, dtype=float)
        self.m, self.n = self.A.shape
        self.p = {**PARAMS_DEFECTO, **params}
        if self.p["p_mut"] is None:
            self.p["p_mut"] = 1.0 / self.n                # ~1 gen mutado por hijo
        self.rng = np.random.default_rng(seed)
        self.cubre = [np.flatnonzero(self.A[:, j]) for j in range(self.n)]  # clientes
```

```
            que cubre cada antena
30
31      # ---------- reparacion ----------
32      def reparar(self, x):
33          x = x.astype(bool).copy()
34          cnt = (self.A @ x.astype(np.float32))               # veces que se cubre cada
                cliente
35          while True:
36              sin = cnt < 0.5
37              if not sin.any():
38                  break
39              ganancia = sin.astype(np.float32) @ self.A      # clientes nuevos que aporta
                    cada antena
40              ganancia[x] = 0
41              idx = np.flatnonzero(ganancia > 0)
42              j = idx[np.argmin(self.c[idx] / ganancia[idx])]
43              x[j] = True
44              cnt[self.cubre[j]] += 1
45          for j in sorted(np.flatnonzero(x), key=lambda k: -self.c[k]):    # eliminar
                redundantes
46              if (cnt[self.cubre[j]] >= 2).all():
47                  x[j] = False
48                  cnt[self.cubre[j]] -= 1
49          return x
50
51      def costo(self, x):
52          return float(self.c @ x)
53
54      # ---------- operadores ----------
55      def seleccion_torneo(self, costos):
56          cand = self.rng.integers(0, len(costos), size=self.p["torneo"])
57          return cand[np.argmin(costos[cand])]
58
59      def cruce_uniforme(self, a, b):
60          mascara = self.rng.random(self.n) < 0.5
61          return np.where(mascara, a, b)
62
63      def mutacion(self, x):
64          flip = self.rng.random(self.n) < self.p["p_mut"]
65          return np.logical_xor(x, flip)
66
67      # ---------- bucle principal ----------
68      def ejecutar(self):
69          P = self.p
70          t0 = time.time()
71          pob = [self.reparar(self.rng.random(self.n) < P["p_init"]) for _ in range(P["pop"
                ])]
72          costos = np.array([self.costo(x) for x in pob])
73          mejor_x, mejor_c = pob[int(np.argmin(costos))].copy(), costos.min()
74          hist_mejor, hist_medio, hist_t = [mejor_c], [costos.mean()], [time.time() - t0]
75          sin_mejora = 0
76          for g in range(1, P["max_gen"] + 1):
77              orden = np.argsort(costos)
78              nueva = [pob[i].copy() for i in orden[:P["n_elite"]]]           # elitismo
79              while len(nueva) < P["pop"]:
80                  a = pob[self.seleccion_torneo(costos)]
81                  b = pob[self.seleccion_torneo(costos)]
82                  hijo = self.cruce_uniforme(a, b) if self.rng.random() < P["p_cruce"] else
                        a.copy()
83                  hijo = self.reparar(self.mutacion(hijo))
84                  nueva.append(hijo)
85              pob = nueva
86              costos = np.array([self.costo(x) for x in pob])
87              if costos.min() < mejor_c - 1e-9:
```

```
                mejor_c, mejor_x, sin_mejora = costos.min(), pob[int(np.argmin(costos))].
                    copy(), 0
            else:
                sin_mejora += 1
            hist_mejor.append(mejor_c); hist_medio.append(costos.mean()); hist_t.append(
                time.time() - t0)
            if sin_mejora >= P["paciencia"]:
                break
            if P["limite_tiempo"] and time.time() - t0 > P["limite_tiempo"]:
                break
        return {"x": mejor_x.astype(int), "costo": float(mejor_c), "tiempo": time.time()
            - t0,
                "generaciones": g, "hist_mejor": hist_mejor, "hist_medio": hist_medio, "
                    hist_t": hist_t}

if __name__ == "__main__":
    from setcover_datos import cargar_datos
    A, c = cargar_datos()
    r = SetCoverGA(A, c, seed=1).ejecutar()
    print("Costo GA:", r["costo"], "| antenas:", int(r["x"].sum()), "| gens:", r["
        generaciones"],
          "| %.1f s" % r["tiempo"], "| valida:", bool((A @ r["x"] >= 1).all()))
```

Apéndice D. Reproducibilidad y repositorio

Todo el código, los datos y los resultados están publicados en GitHub: https://github.com/IsabelVargas2/set-cover-antenas. Para reproducir los resultados:

1. Instalar las dependencias: `pip install -r requirements.txt`.
2. Resolver con el método exacto: `python correr_exacto.py`.
3. Ejecutar los experimentos del Algoritmo Genético: `python experimentos_ga.py` y `python experimento_extendido.py`.
4. Generar las figuras: `python hacer_figuras.py`.
5. Abrir la interfaz visual: `streamlit run app.py`.

Los tiempos dependen del equipo; los costos del Algoritmo Genético dependen de la semilla, que se fija en cada corrida para que los resultados se puedan repetir.

References

[1] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, Eds. New York: Plenum Press, 1972, pp. 85–103.

[2] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: W. H. Freeman, 1979.

[3] V. Chvátal, "A greedy heuristic for the set-covering problem," *Mathematics of Operations Research*, vol. 4, no. 3, pp. 233–235, 1979.

[4] U. Feige, "A threshold of $\ln n$ for approximating set cover," *Journal of the ACM*, vol. 45, no. 4, pp. 634–652, 1998.

[5] J. E. Beasley and P. C. Chu, "A genetic algorithm for the set covering problem," *European Journal of Operational Research*, vol. 94, no. 2, pp. 392–404, 1996.

[6] A. Caprara, M. Fischetti, and P. Toth, "Algorithms for the set covering problem," *Annals of Operations Research*, vol. 98, pp. 353–371, 2000.

[7] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor: University of Michigan Press, 1975.

[8] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Reading, MA: Addison-Wesley, 1989.

[9] A. E. Eiben and J. E. Smith, *Introduction to Evolutionary Computing*, 2nd ed. Berlin: Springer, 2015.

[10] L. A. Wolsey, *Integer Programming*. New York: Wiley, 1998.

[11] A. H. Land and A. G. Doig, "An automatic method of solving discrete programming problems," *Econometrica*, vol. 28, no. 3, pp. 497–520, 1960.

[12] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.

[13] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.